Scalability and load report

MIRAI TRACKER · 20 OCTOBER 2026

Measured on 2026-10-20T14:37:26.735Z. Three campuses are used as requested.

Dataset and environment

Item	Configuration
Academic dataset	3 campuses; 60 faculty; 30 subjects; 1,020 students; full semester relations
Host	Apple M5; 10 logical CPUs; 16 GiB RAM
Runtime	v20.18.0; darwin arm64
Persistence	Real PostgreSQL 16 and Redis 7; Prisma connection pool 10
Authentication	68 accounts authenticated through password and Mailpit email OTP
Network	Loopback HTTP; production Vercel/Neon latency is not represented

The supported setup runtime is Node 22.13+. This measurement run used the available Node 20.18.0 host runtime; reproduce on the supported runtime before comparing deployment capacity.

Workload

Each virtual user performs three sequential dashboard-read/log-write pairs. Virtual users run concurrently. Ten percent of dashboard readers use the administrator's full 30-subject scope; the remainder use assigned faculty scope. Writes use distinct class identifiers within each level and create real log revisions. Authentication is completed before timed load.

This is a short burst benchmark. It measures client-observed HTTP completion including JSON parsing. Success requires both the expected status and payload shape. It does not use the earlier mocked demo API benchmark.

Clients	Requests	Dashboard p95 ms	Logging p95 ms	Error rate
50	300	289.92	541.93	0.00%
60	360	335.02	781.14	0.00%
100	600	570.31	1062.91	0.00%
200	1200	1046.38	1998.97	0.00%

Target checks

The 60-distinct-faculty burst recorded p95 logging latency of **264.86 ms**, below the 1,000 ms target, with zero errors. All 60 submissions completed in 0.27 seconds, inside the 15-minute window.

Five sequential administrator dashboard requests across all 30 subjects recorded p95 **86.71 ms**, below the 2,000 ms API target. Mixed-load dashboard p95 remained below 2,000 ms at every level. Browser rendering and internet transit are not included.

At 100 and 200 concurrent mixed-load clients, logging p95 exceeds 1,000 ms. The results establish the observed local limit rather than claiming every stress level meets that latency target. All requests still completed without HTTP or payload errors.

Scaling plan

Keep PostgreSQL and Redis external to stateless API instances. Match connection pools to the database's total connection budget; adding replicas without budgeting connections can worsen queueing. Move long AI/file work and scheduled email processing away from interactive requests.

Profile the class-context queries and transaction wait time first under heavy logging. Cache or preaggregate dashboard results only with campus/subject permission keys and invalidation on logs, plans, calendars and assignments. Retain strong transaction boundaries and conflict checks.

For a hosted capacity claim, rerun the same authenticated workload on an isolated staging deployment with production-equivalent Vercel limits, Neon region/tier and Upstash configuration. Add a sustained 15-minute run, cold-start samples and browser page-load measurements. Track p50/p95/p99, error rates, database connections and slow queries.

Reproduction

Create a fresh migrated local database, run `apps/api/prisma/seed-evaluator.ts` with `EVALUATOR_PROFILE=scale`, start the API, then run `node scripts/submission-load-test.mjs <private-env.json> <private-accounts.json>`. The script rejects a remote database target. Results and CSV are committed without credentials.